

```
[1] import pycaret
from pycaret.datasets import get_data
data = get_data('diabetes')
```

	Number of times pregnant	Plasma glucose concentration a 2 hours in an oral glucose tolerance test	Diastolic blood pressure (mm Hg)	Triceps skin fold thickness (mm)	2-Hour serum insulin (mu U/ml)	Body mass index (weight in kg/(height in m) ²)	Diabetes pedigree function	Age (years)	Class variable
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Start coding or generate with AI.

Figure 1: Loading the dataset

```
data = get_data('diabetes')
```

In this dataset we have eight features or columns, and the label column is the 'Class' variable as shown in Figure 1.

In a data science lifecycle, once the dataset is ready, the next step is to perform the data preprocessing. This includes data cleaning, taking care of the null or empty values, encoding the category label variables, data scaling, and a few other things. In our case PyCaret takes care of all of this by using the following lines of code as shown in Figure 2.

```
from pycaret.classification import *
experiment = setup(data, target='Class variable', session_id=123)
```

The next step is to choose the best algorithm for our use case, which is to predict if a person will get diabetes based on a set of features. Typically, data scientists do a little trial and error on a few algorithms that they believe will work for the use case. This step is also taken care of by PyCaret. A comparison of algorithms can be done using the following line of code as shown in Figure 3.

```
best_model = compare_models()
```

This is a very interesting session where you get to see the progress as PyCaret goes through each algorithm. This tool also highlights the best parts of each algorithm, in terms of training time, accuracy, precision, recall, etc.

```
from pycaret.classification import *
experiment = setup(data, target='Class variable', session_id=123)
```

	Description	Value
0	Session id	123
1	Target	Class variable
2	Target type	Binary
3	Original data shape	(768, 9)
4	Transformed data shape	(768, 9)
5	Transformed train set shape	(537, 9)
6	Transformed test set shape	(231, 9)
7	Numeric features	8
8	Preprocess	True
9	Imputation type	simple
10	Numeric imputation	mean
11	Categorical imputation	mode
12	Fold Generator	StratifiedKFold
13	Fold Number	10
14	CPU jobs	-1
15	Use GPU	False
16	Log Experiment	False
17	Experiment Name	clf-default-name
18	USI	878d

Figure 2: Preprocessing data

```
best_model = compare_models()
```

	Model	Accuracy	AUC	Recall	Prec.	F1	Kappa	MCC	TT (Sec)
lr	Logistic Regression	0.7689	0.8047	0.5602	0.7208	0.6279	0.4691	0.4736	0.6020
ridge	Ridge Classifier	0.7670	0.8060	0.5497	0.7234	0.6221	0.4581	0.4690	0.0520
lda	Linear Discriminant Analysis	0.7670	0.8055	0.5550	0.7202	0.6243	0.4594	0.4695	0.0270
rf	Random Forest Classifier	0.7485	0.7911	0.5284	0.6811	0.5924	0.4150	0.4238	0.2140
nb	Naive Bayes	0.7427	0.7955	0.5702	0.6543	0.6043	0.4156	0.4215	0.0450
gbc	Gradient Boosting Classifier	0.7373	0.7909	0.5550	0.6445	0.5931	0.4013	0.4059	0.1690
ada	Ada Boost Classifier	0.7372	0.7799	0.5275	0.6585	0.5796	0.3926	0.4017	0.1250
et	Extra Trees Classifier	0.7299	0.7788	0.4965	0.6516	0.5596	0.3706	0.3802	0.2940
qda	Quadratic Discriminant Analysis	0.7282	0.7894	0.5281	0.6558	0.5736	0.3785	0.3910	0.0290
lightgbm	Light Gradient Boosting Machine	0.7133	0.7645	0.5398	0.6036	0.5650	0.3534	0.3580	0.5200
knn	K Neighbors Classifier	0.7001	0.7164	0.5020	0.5982	0.5413	0.3209	0.3271	0.0750
dt	Decision Tree Classifier	0.6928	0.6512	0.5137	0.5636	0.5328	0.3070	0.3098	0.0490
xgboost	Extreme Gradient Boosting	0.6891	0.7572	0.5292	0.5668	0.5438	0.3089	0.3122	0.1840
dummy	Dummy Classifier	0.6518	0.5000	0.0000	0.0000	0.0000	0.0000	0.0000	0.0260
svm	SVM - Linear Kernel	0.5954	0.5914	0.3395	0.4090	0.2671	0.0720	0.0912	0.0510

Figure 3: Comparing various algorithms